

REPUBLIQUE DU SENEGAL



Un peuple-un but-une foi

Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation

Direction de l'Enseignement Supérieur Privé

Institut Supérieur d'Informatique

ISI

Master Professionnel en Réseaux et Systèmes Informatiques

RAPPORT DE FIN DE MODULE

Module : Programmation et Automatisation des Réseaux (DevNet Associate)

Automatisation complète d'un réseau campus multi-sites

Présenté par :

Mohamed Abdallahi Abdoulaye Diallo

Sous la direction de :

M. Baye Sabarane LAM

Année académique 2026-2027

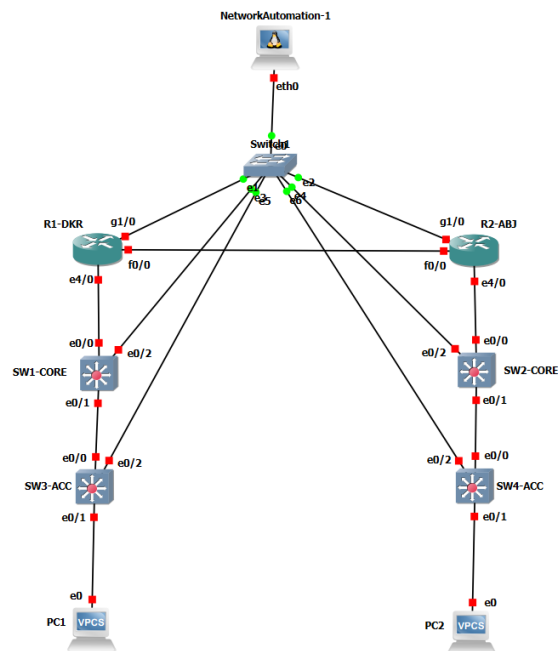
Introduction

Dans le cadre du déploiement et de la gestion des infrastructures réseaux modernes, l'automatisation est devenue un levier indispensable pour garantir la rapidité, la fiabilité et la traçabilité des opérations. La configuration manuelle équipement par équipement via une interface en ligne de commande (CLI) présente de nombreux risques d'erreurs humaines, d'hétérogénéité des configurations et entraîne une perte de temps conséquente, particulièrement sur des topologies multi-sites.

Ce projet a pour objectif d'automatiser le processus de déploiement et de validation des configurations sur l'ensemble des équipements de notre topologie (routeurs WAN, commutateurs de cœur et d'accès). À l'aide d'un script développé en Python et basé sur la bibliothèque **Netmiko**, nous avons mis en place une chaîne d'exécution automatisée permettant l'autodétection des OS, la sauvegarde préventive des états réseaux (*pre-check backup*), l'application dynamique des configurations et la vérification de la connectivité de bout en bout.

Introduction	2
1 La topologie	4
2 Configuration Initiale de Management	4
3 Préparation du Conteneur d'Automatisation	7
4 Structure du Projet & Fichiers de Configuration	9
4.2 Créer les Fichiers de Configuration Réseau (configs/)	9
4.3 Test de ping.....	12
4.4 Vérification avec telnet.....	13
5.3 Déroulement des Instructions du Script	15
5.5 Chargement et application des configurations	15
5.6 Sauvegarde en NVRAM et traçabilité (logging).....	16
5.7 Exécution du script.....	16
5.8 Rencontre dans problème de config.....	16
5.9 Réexécution le script.....	20
6 Configurer PC1 (VLAN 10 - Dakar)	20
6.1 Tester la connectivité locale (Passerelles).....	21
6.2 Tester de la connectivité globale	23
6.3 Vérification de ping	26
Conclusion.....	28

1 La topologie



2 Configuration Initiale de Management

Cette étape se fait manuellement sur la console de chaque équipement. Elle active le serveur SSH et attribue une adresse IP sur le réseau de management

R1-DKR

Interface management vers Switch-1 : g1/0

```
R1-DKR(config)#enable password cisco
R1-DKR(config)#username devnet privilege 15 password 0 cisco
R1-DKR(config)#interface g1/0
R1-DKR(config-if)# ip address 192.168.10.10 255.255.255.0
R1-DKR(config-if)# no shut
R1-DKR(config-if)#line con 0
R1-DKR(config-line)# password cisco
R1-DKR(config-line)#line aux 0
R1-DKR(config-line)#line vty 0 4
R1-DKR(config-line)# exec-timeout 720 0
R1-DKR(config-line)# login local
R1-DKR(config-line)# password cisco
R1-DKR(config-line)# transport input all
R1-DKR(config-line)#end
R1-DKR#wr memory
*Sep  7 18:38:03.615: %SYS-5-CONFIG_I: Configured from console by console
R1-DKR#wr memory
*Sep  7 18:38:05.479: %LINK-3-UPDOWN: Interface GigabitEthernet1/0, changed state to up
*Sep  7 18:38:06.479: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet1/0, changed state to up
R1-DKR#wr memory
Warning: Attempting to overwrite an NVRAM configuration previously written
by a different version of the system image.
Overwrite the previous NVRAM configuration?[confirm]
Building configuration...
[OK]
R1-DKR#
```

R2-ABJ

Interface management vers Switch-1 : g1/0

```
R2-ABJ(config)#enable password cisco
R2-ABJ(config)#username devnet privilege 15 password 0 cisco
R2-ABJ(config)#interface g1/0
R2-ABJ(config-if)# ip address 192.168.10.20 255.255.255.0
R2-ABJ(config-if)# no shut
R2-ABJ(config-if)#line con 0
R2-ABJ(config-line)# password cisco
R2-ABJ(config-line)#line aux 0
R2-ABJ(config-line)#line vty 0 4
R2-ABJ(config-line)# exec-timeout 720 0
R2-ABJ(config-line)# login local
R2-ABJ(config-line)# password cisco
R2-ABJ(config-line)# transport input all
R2-ABJ(config-line)#end
R2-ABJ#wr memory
*Sep  7 18:39:33.295: %SYS-5-CONFIG_I: Configured from console by console
R2-ABJ#wr memory
Warning: Attempting to overwrite an NVRAM configuration previously written
by a different version of the system image.
Overwrite the previous NVRAM configuration?[confirm]
*Sep  7 18:39:35.187: %LINK-3-UPDOWN: Interface GigabitEthernet1/0, changed state to up
*Sep  7 18:39:36.187: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet1/0, changed state to up
[confirm]
Building configuration...
[OK]
R2-ABJ#
```

SW1-CORE (Switch Cœur Dakar)

Interface management vers Switch-1 : e0/2

```

Enter configuration commands, one per line. End with CNTL/Z.
SW1-CORE(config)#enable password cisco
SW1-CORE(config)#username devnet privilege 15 password 0 cisco
SW1-CORE(config)#interface Vlan1
SW1-CORE(config-if)# ip address 192.168.10.30 255.255.255.0
SW1-CORE(config-if)# no shut
SW1-CORE(config-if)#ip default-gateway 10.10.10.1
SW1-CORE(config)#line con 0
SW1-CORE(config-line)# password cisco
SW1-CORE(config-line)#line aux 0
SW1-CORE(config-line)#line vty 0 4
SW1-CORE(config-line)# exec-timeout 720 0
SW1-CORE(config-line)# login local
SW1-CORE(config-line)# password cisco
SW1-CORE(config-line)# transport input all
SW1-CORE(config-line)#end
SW1-CORE#wr memory
*Sep  7 18:40:36.772: %SYS-5-CONFIG_I: Configured from console by console
SW1-CORE#wr memory
*Sep  7 18:40:38.770: %LINK-3-UPDOWN: Interface Vlan1, changed state to up
*Sep  7 18:40:39.775: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan1, changed state to up
SW1-CORE#wr memory
Warning: Attempting to overwrite an NVRAM configuration previously written
by a different version of the system image.
Overwrite the previous NVRAM configuration?[confirm]
Building configuration...
Compressed configuration from 1640 bytes to 1004 bytes[OK]
SW1-CORE#

```

SW2-CORE (Switch Cœur Abidjan)

Interface management vers Switch-1 : e0/2

```

SW2-CORE(config)#enable password cisco
SW2-CORE(config)#username devnet privilege 15 password 0 cisco
SW2-CORE(config)#interface Vlan1
SW2-CORE(config-if)# ip address 192.168.10.40 255.255.255.0
SW2-CORE(config-if)# no shut
SW2-CORE(config-if)#ip default-gateway 10.20.10.1
SW2-CORE(config)#line con 0
SW2-CORE(config-line)# password cisco
SW2-CORE(config-line)#line aux 0
SW2-CORE(config-line)#line vty 0 4
SW2-CORE(config-line)# exec-timeout 720 0
SW2-CORE(config-line)# login local
SW2-CORE(config-line)# password cisco
SW2-CORE(config-line)# transport input all
SW2-CORE(config-line)#end
SW2-CORE#wr memory
Warning: Attempting to overwrite an NVRAM configuration previously written
by a different version of the system image.
Overwrite the previous NVRAM configuration?[confirm]
*Sep  7 18:41:39.144: %SYS-5-CONFIG_I: Configured from console by console
[confirm]
Building configuration...
Compressed configuration from 1640 bytes to 1006 bytes[OK]
SW2-CORE#
*Sep  7 18:41:41.141: %LINK-3-UPDOWN: Interface Vlan1, changed state to up
*Sep  7 18:41:42.146: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan1, changed state to up
SW2-CORE#

```

SW3-ACC (Switch Accès Dakar)

Interface management vers Switch-1 : e0/2

```

SW3-ACC(config)#enable password cisco
SW3-ACC(config)#username devnet privilege 15 password 0 cisco
SW3-ACC(config)#interface Vlan1
SW3-ACC(config-if)# ip address 192.168.10.50 255.255.255.0
SW3-ACC(config-if)# no shut
SW3-ACC(config-if)#ip default-gateway 10.10.10.1
SW3-ACC(config)#line con 0
SW3-ACC(config-line)# password cisco
SW3-ACC(config-line)#line aux 0
SW3-ACC(config-line)#line vty 0 4
SW3-ACC(config-line)# exec-timeout 720 0
SW3-ACC(config-line)# login local
SW3-ACC(config-line)# password cisco
SW3-ACC(config-line)# transport input all
SW3-ACC(config-line)#end
SW3-ACC#wr memory
Warning: Attempting to overwrite an NVRAM configuration previously written
by a different version of the system image.
Overwrite the previous NVRAM configuration?[confirm]
*Sep  7 18:42:47.221: %SYS-5-CONFIG_I: Configured from console by console
[confirm]
Building configuration...
Compressed configuration from 1639 bytes to 1003 bytes[OK]
SW3-ACC#
*Sep  7 18:42:49.220: %LINK-3-UPDOWN: Interface Vlan1, changed state to up
*Sep  7 18:42:50.228: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan1, changed state to up
SW3-ACC#

```

SW4-ACC (Switch Accès Abidjan)

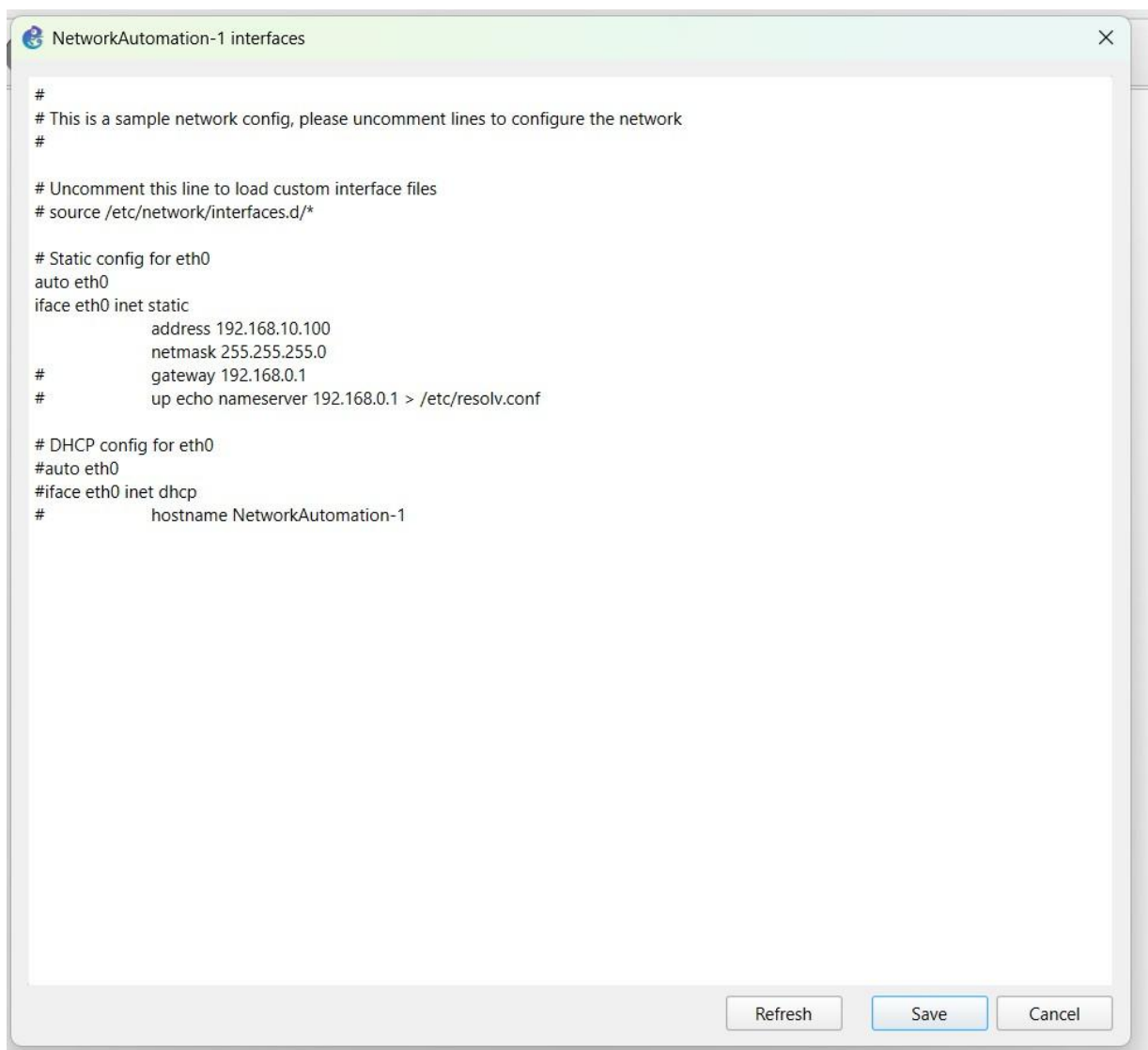
Interface management vers Switch-1 : e0/2

```

SW4-ACC(config)#enable password cisco
SW4-ACC(config)#username devnet privilege 15 password 0 cisco
SW4-ACC(config)#interface Vlan1
SW4-ACC(config-if)# ip address 192.168.10.60 255.255.255.0
SW4-ACC(config-if)# no shut
SW4-ACC(config-if)#ip default-gateway 10.20.10.1
SW4-ACC(config)#line con 0
SW4-ACC(config-line)# password cisco
SW4-ACC(config-line)#line aux 0
SW4-ACC(config-line)#line vty 0 4
SW4-ACC(config-line)# exec-timeout 720 0
SW4-ACC(config-line)# login local
SW4-ACC(config-line)# password cisco
SW4-ACC(config-line)# transport input all
SW4-ACC(config-line)#end
SW4-ACC#wr memory
Warning: Attempting to overwrite an NVRAM configuration previously written
by a different version of the system image.
Overwrite the previous NVRAM configuration?[confirm]
*Sep  7 18:43:53.056: %SYS-5-CONFIG_I: Configured from console by console
[confirm]
Building configuration...
Compressed configuration from 1639 bytes to 1006 bytes[OK]
SW4-ACC#
*Sep  7 18:43:55.055: %LINK-3-UPDOWN: Interface Vlan1, changed state to up
*Sep  7 18:43:56.059: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan1, changed state to up
SW4-ACC#

```

3 Préparation du Conteneur d'Automatisation



On fixe l'adresse à 192.168.10.100

```
root@NetworkAutomation-1:~# ifconfig  
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 192.168.10.100 netmask 255.255.255.0 broadcast 0.0.0.0  
    inet6 fe80::42:3bff:fe01:8800 prefixlen 64 scopeid 0x20<link>  
    ether 02:42:3b:01:88:00 txqueuelen 1000 (Ethernet)  
    RX packets 0 bytes 0 (0.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 7 bytes 586 (586.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 0 bytes 0 (0.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 0 bytes 0 (0.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
root@NetworkAutomation-1:~#
```

On voit que l'adresse a été bien attribuée à la machine

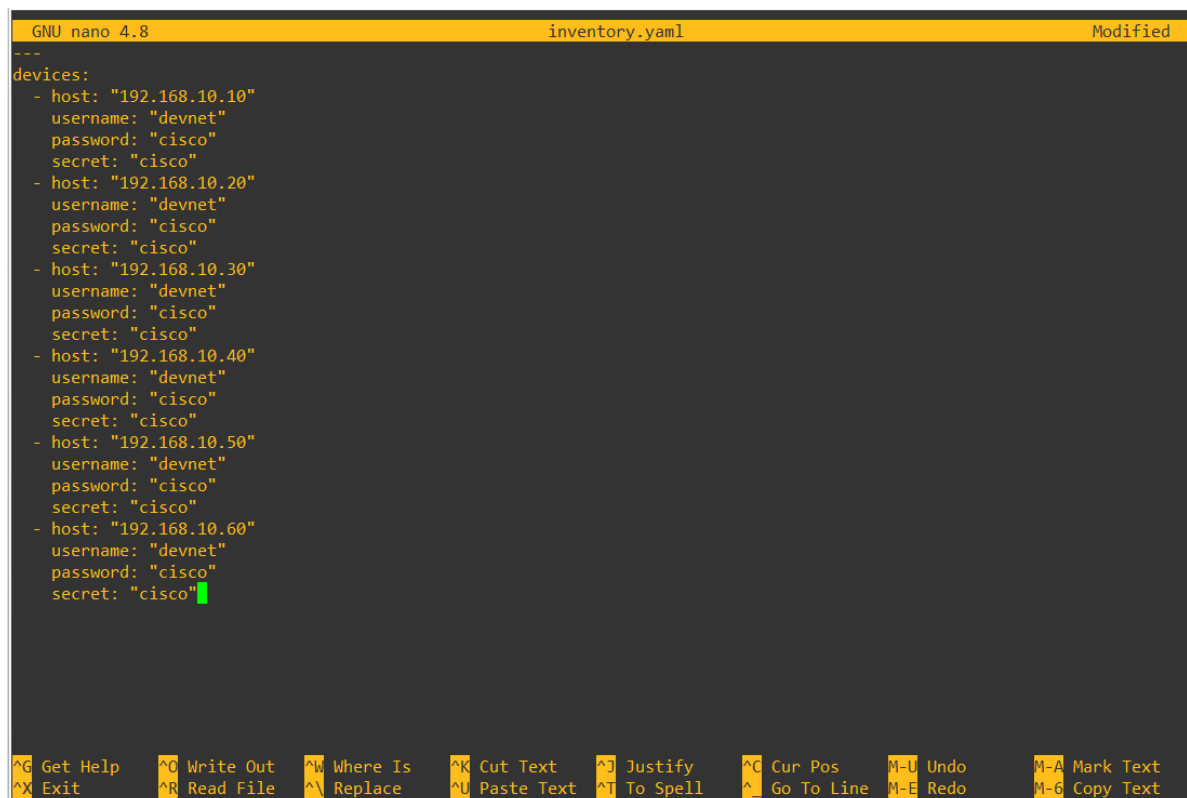
4 Structure du Projet & Fichiers de Configuration

4.1 Créez le projet sur la machine NetworkAutomation-1

```
root@NetworkAutomation-1:~# mkdir -p projet-devnet-campus/{src,configs,logs,backups,tests}
root@NetworkAutomation-1:~# cd projet-devnet-campus
root@NetworkAutomation-1:~/projet-devnet-campus# ls
backups configs logs src tests
root@NetworkAutomation-1:~/projet-devnet-campus#
```

4.2 Créer les Fichiers de Configuration Réseau (configs/)

D'abord on fait un Inventaire des équipements entièrement externalisé dans un fichier inventory.yaml



```
GNU nano 4.8      inventory.yaml      Modified
---
devices:
- host: "192.168.10.10"
  username: "devnet"
  password: "cisco"
  secret: "cisco"
- host: "192.168.10.20"
  username: "devnet"
  password: "cisco"
  secret: "cisco"
- host: "192.168.10.30"
  username: "devnet"
  password: "cisco"
  secret: "cisco"
- host: "192.168.10.40"
  username: "devnet"
  password: "cisco"
  secret: "cisco"
- host: "192.168.10.50"
  username: "devnet"
  password: "cisco"
  secret: "cisco"
- host: "192.168.10.60"
  username: "devnet"
  password: "cisco"
  secret: "cisco"
```

Pour correspondre exactement à ces adresses IP et aux identifiants créés (devnet / cisco), on met à jour le fichier YAML sur la machine d'automatisation

Maintenant on crée les fichiers des configurations réseaux

R1-DKR.txt

```
GNU nano 4.8 R1-DKR.txt
interface FastEthernet4/0
description Link to SW1-CORE
no shutdown
!
interface FastEthernet4/0.10
description Gateway LAN DKR
encapsulation dot1Q 10
ip address 10.10.10.1 255.255.255.0
!
interface FastEthernet0/0
description WAN Link to R2-ABJ
ip address 192.168.20.1 255.255.255.0
no shutdown
!
router eigrp 100
network 10.10.10.0 0.0.0.255
network 192.168.20.0 0.0.0.255
eigrp router-id 1.1.1.1

[ Read 18 lines ]
^G Get Help      ^O Write Out    ^W Where Is     ^K Cut Text     ^J Justify      ^C Cur Pos      M-U Undo        M-A Mark Text
^X Exit          ^R Read File    ^\ Replace      ^U Paste Text   ^T To Spell     ^_ Go To Line    M-E Redo        M-G Copy Text
```

R2-ABJ.txt

```
GNU nano 4.8 configs/R2-ABJ.txt
interface FastEthernet4/0
description Link to SW2-CORE
no shutdown
!
interface FastEthernet4/0.20
description Gateway LAN ABJ
encapsulation dot1Q 20
ip address 10.20.10.1 255.255.255.0
!
interface FastEthernet0/0
description WAN Link to R1-DKR
ip address 192.168.20.2 255.255.255.0
no shutdown
!
router eigrp 100
network 10.20.10.0 0.0.0.255
network 192.168.20.0 0.0.0.255
eigrp router-id 2.2.2.2

[ Read 18 lines ]
^G Get Help      ^O Write Out    ^W Where Is     ^K Cut Text     ^J Justify      ^C Cur Pos      M-U Undo        M-A Mark Text
^X Exit          ^R Read File    ^\ Replace      ^U Paste Text   ^T To Spell     ^_ Go To Line    M-E Redo        M-G Copy Text
```

SW1-CORE.txt

```
GNU nano 4.8                                configs/SW1-CORE.txt                                Modified
vlan 10
name LAN-DKR
!
interface Ethernet0/0
description Trunk to R1-DKR
switchport mode trunk
switchport trunk allowed vlan 10
no shutdown
!
interface Ethernet0/1
description Trunk to SW3-ACC
switchport mode trunk
switchport trunk allowed vlan 10
no shutdown
```

SW2-CORE.txt

```
GNU nano 4.8                                configs/SW2-CORE.txt                                Modified
vlan 20
name LAN-ABJ
!
interface Ethernet0/0
description Trunk to R2-ABJ
switchport mode trunk
switchport trunk allowed vlan 20
no shutdown
!
interface Ethernet0/1
description Trunk to SW4-ACC
switchport mode trunk
switchport trunk allowed vlan 20
no shutdown
```

SW3-ACC.txt

```
GNU nano 4.8                                configs/SW3-ACC.txt                                Modified
vlan 10
name LAN-DKR
!
interface Ethernet0/0
description Trunk to SW1-CORE
switchport mode trunk
switchport trunk allowed vlan 10
no shutdown
!
interface Ethernet0/1
description Access to PC1
switchport mode access
switchport access vlan 10
spanning-tree portfast
no shutdown
```

GNU nano 4.8 shows a configuration file named configs/SW3-ACC.txt. The configuration defines VLAN 10 named LAN-DKR. It configures interface Ethernet0/0 as a trunk port for VLAN 10 and interface Ethernet0/1 as an access port for VLAN 10 with spanning-tree portfast. The bottom status bar shows various nano editor shortcuts like ^G Get Help, ^O Write Out, ^W Where Is, ^K Cut Text, ^J Justify, ^C Cur Pos, M-U Undo, M-A Mark Text, ^X Exit, ^R Read File, ^_ Replace, ^U Paste Text, ^T To Spell, ^_ Go To Line, M-E Redo, and M-G Copy Text.

SW4-ACC.txt

```
GNU nano 4.8                                configs/SW4-ACC.txt                                Modified
vlan 20
name LAN-ABJ
!
interface Ethernet0/0
description Trunk to SW2-CORE
switchport mode trunk
switchport trunk allowed vlan 20
no shutdown
!
interface Ethernet0/1
description Access to PC2
switchport mode access
switchport access vlan 20
spanning-tree portfast
no shutdown
```

GNU nano 4.8 shows a configuration file named configs/SW4-ACC.txt. The configuration defines VLAN 20 named LAN-ABJ. It configures interface Ethernet0/0 as a trunk port for VLAN 20 and interface Ethernet0/1 as an access port for VLAN 20 with spanning-tree portfast. The bottom status bar shows various nano editor shortcuts.

4.3 Test de ping

Après avoir fait toutes les configurations nécessaires, sur La machine automation J'ai fait un ping vers R1-DKR J'ai fait aussi l'inverse

```

root@NetworkAutomation-1:~# ping -c 2 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data.
64 bytes from 192.168.10.10: icmp_seq=1 ttl=255 time=43.7 ms
64 bytes from 192.168.10.10: icmp_seq=2 ttl=255 time=13.6 ms

--- 192.168.10.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 13.564/28.623/43.683/15.059 ms

```

```

R1-DKR#ping 192.168.10.100
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.10.100, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 12/16/24 ms
R1-DKR#

```

On voit que les deux Équipements Arrivent à communiquer ce qui prouve que les configurations sont appliquées avec succès c'est

4.4 Vérification avec telnet

```

root@NetworkAutomation-1:~/projet-devnet-campus# telnet 192.168.10.10
Trying 192.168.10.10...
Connected to 192.168.10.10.
Escape character is '^]'.

User Access Verification

Username: devnet
% Username: timeout expired!
Username:
Username: devnet
Password:
R1-DKR#exit
Connection closed by foreign host.

```

Sur la machine automation j'ai fait une connexion telle sur le routeur R1-DKR La connexion passe Le terminal me renvoie le prompt du R1-DKR

5 Script d'Automatisation (src/main.py)

5.1 Créer le fichier src/main.py

Sur la machine automation Je crée Le fichier main.py qui va contenir le script avec les Instructions qui doivent être appliquées sur les 6 équipements

```
GNU nano 4.8 src/main.py Modified
import os
import yaml
import logging
from datetime import datetime
from netmiko import ConnectHandler, SSHDetect
from netmiko.exceptions import NetmikoTimeoutException, NetmikoAuthenticationException

logging.basicConfig(
    filename='logs/deployment.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

def load_inventory(filepath="inventory.yaml"):
    with open(filepath, "r") as f:
        return yaml.safe_load(f)["devices"]

def get_role_from_prompt(prompt):
    hostname = prompt.rstrip("#").strip()
    if hostname.startswith("R"):
        return "routeur", hostname
    elif "CORE" in hostname:
        return "core", hostname
    elif "ACC" in hostname:
        return "accs", hostname
    return "inconnu", hostname

def load_config_commands(hostname):
    path = f'configs/{hostname}.txt'
    if not os.path.exists(path):
        raise FileNotFoundError(f"Fichier de configuration introuvable : {path}")
    with open(path, "r", encoding="utf-8") as f:
        return [line.strip() for line in f if line.strip() and not line.strip().startswith("#")]

def deploy():
    os.makedirs("backups", exist_ok=True)
    os.makedirs("logs", exist_ok=True)
    devices = load_inventory()

    for dev in devices:
        ip = dev["host"]
        print(f"Connexion vers {ip}...")

        try:
            # 1. Autodetection du type OS avec SSHDetect
            guesser = SSHDetect()
            dev["device_type"] = guesser.autodetect()
            print(f"OS détecté : {dev['device_type']}")

            with ConnectHandler(**dev) as net:
                net.enable()

                # 2. Identification dynamique via le prompt
                prompt = net.find_prompt()
                role, hostname = get_role_from_prompt(prompt)
                print(f"Équipement : {hostname} | Rôle : {role}")

                # 3. Sauvegarde pré-déploiement
                timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
                backup_filename = f"backups/{hostname}.{timestamp}.cfg"
                running_config = net.send_command("show running-config")
                with open(backup_filename, "w") as bf:
                    bf.write(running_config)
                print(f"Backup sauvegardé : {backup_filename}")

                # 4. Application des configurations
                commands = load_config_commands(hostname)
                output = net.send_config_set(commands)
                print(f"Configuration appliquée avec succès sur {hostname}.")

                # 5. Écriture en NVRAM
                net.save_config()
                logging.info(f"Déploiement réussi sur {hostname} ({ip})")

            except NetmikoAuthenticationException:
                err = f"Échec d'authentification sur {ip}"
                print(f"Erreur : {err}")
                logging.error(err)
            except NetmikoTimeoutException:
                err = f"Timeout / Équipement injoignable : {ip}"
                print(f"Erreur : {err}")
                logging.error(err)
            except Exception as e:
                err = f"Erreur sur {ip} : {str(e)}"
                print(f"Erreur : {err}")
                logging.error(err)

if __name__ == "__main__":
    deploy()
```

```
with ConnectHandler(**dev) as net:
    net.enable()

    # 2. Identification dynamique via le prompt
    prompt = net.find_prompt()
    role, hostname = get_role_from_prompt(prompt)
    print(f"Équipement : {hostname} | Rôle : {role}")

    # 3. Sauvegarde pré-déploiement
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    backup_filename = f"backups/{hostname}.{timestamp}.cfg"
    running_config = net.send_command("show running-config")
    with open(backup_filename, "w") as bf:
        bf.write(running_config)
    print(f"Backup sauvegardé : {backup_filename}")

    # 4. Application des configurations
    commands = load_config_commands(hostname)
    output = net.send_config_set(commands)
    print(f"Configuration appliquée avec succès sur {hostname}.")

    # 5. Écriture en NVRAM
    net.save_config()
    logging.info(f"Déploiement réussi sur {hostname} ({ip})")

except NetmikoAuthenticationException:
    err = f"Échec d'authentification sur {ip}"
    print(f"Erreur : {err}")
    logging.error(err)
except NetmikoTimeoutException:
    err = f"Timeout / Équipement injoignable : {ip}"
    print(f"Erreur : {err}")
    logging.error(err)
except Exception as e:
    err = f"Erreur sur {ip} : {str(e)}"
    print(f"Erreur : {err}")
    logging.error(err)

if __name__ == "__main__":
    deploy()
```

Voici tout ce qui contient le script dans le fichier main.py

5.2 Présentation du Script (`main.py`)

Le script d'automatisation vise à sécuriser, accélérer et standardiser le déploiement des configurations sur l'ensemble des équipements de la topologie (routeurs WAN, commutateurs de cœur et d'accès).

Il s'appuie sur la bibliothèque **Netmiko** pour l'interconnexion SSH et s'articule autour de cinq étapes clés exécutées séquentiellement pour chaque équipement du fichier d'inventaire (`inventory.yaml`)

5.3 Déroulement des Instructions du Script

- Auto-détection du système d'exploitation (`SSHDetect`)

Afin d'éviter tout échec lié à un type d'équipement incorrectement déclaré, le script utilise le module `SSHDetect`. Il se connecte à l'adresse IP cible et analyse la réponse de la bannière SSH pour identifier automatiquement l'OS

- Identification dynamique de l'équipement et de son rôle

Une fois la session SSH établie et le mode privilège activé (`enable`), le script lit l'invite de commande (`prompt`) de l'équipement :

- La fonction `get_role_from_prompt` extrait le nom d'hôte (`hostname`).
- Le rôle (Routeur, Switch Cœur, Switch Accès) est automatiquement déduit selon le préfixe ou la nomenclature du nom (ex. `R1-DKR ---Routeur`, `SW1-CORE Cœur`).

5.4 Sauvegarde pré-déploiement (*Pre-check Backup*)

Avant d'appliquer la moindre modification, le script exécute la commande `show running-config` et enregistre l'état actuel de l'équipement dans le dossier `backups/`. Le fichier est horodaté au format `HOSTNAME_YYYYMMDD_HHMMSS.cfg` (ex. `R1-DKR_20260908_143000.cfg`), garantissant une option de **rollback** rapide en cas d'incident.

5.5 Chargement et application des configurations

Le script fait appel à la fonction `load_config_commands` pour lire le fichier de commandes propre à l'équipement dans le dossier `configs/` (ex. `configs/R1-DKR.txt`) :

- Les lignes vides, les commentaires (`#` ou `!`) ainsi que les commandes de passage en mode configuration globale (`configure terminal`, `end`) sont automatiquement nettoyées pour éviter toute erreur de syntaxe.
- La liste des commandes purguées est transmise à `net.send_config_set()`, qui les applique en mode configuration sur l'équipement.

5.6 Sauvegarde en NVRAM et traçabilité (logging)

- **Persistance** : La commande `net.save_config()` est exécutée pour enregistrer la *running-config* vers la *startup-config* (`copy running-config startup-config`), assurant le maintien des configurations après un redémarrage.
- **Traçabilité** : Chaque succès ou échec (erreur d'authentification, timeout SSH) est consigné dans le fichier de journalisation `logs/deployment.log` avec horodatage pour faciliter l'audit.

Bilan de l'Exécution

Grâce à ce script, l'intégralité du parc d'équipements est configurée de manière homogène, sans intervention manuelle répétitive dans les terminaux, tout en garantissant un suivi rigoureux des sauvegardes et des événements d'erreur.

5.7 Exécution du script

```
root@NetworkAutomation-1:~/projet-devnet-campus# python3 src/main.py
Traceback (most recent call last):
  File "src/main.py", line 6, in <module>
    from netmiko.exceptions import NetmikoTimeoutException, NetmikoAuthenticationException
ModuleNotFoundError: No module named 'netmiko.exceptions'
```

En exécutant le script il m'envoie ces informations qu'il y a une erreur sur la ligne 6

L'erreur `ModuleNotFoundError: No module named 'netmiko.exceptions'` se produit en raison d'un changement dans les versions récentes de Netmiko (version 4.x+), où les exceptions ont été déplacées directement à la racine du module (`netmiko.NetmikoTimeoutException` au lieu de `netmiko.exceptions`).

5.8 Rencontre dans problème de config

```
GNU nano 4.8 src/main.py Modified
import os
import yaml
import logging
from datetime import datetime
from netmiko import ConnectHandler, SSHDetect
from netmiko.exceptions import NetmikoTimeoutException, NetmikoAuthenticationException

logging.basicConfig(
    filename='logs/deployment.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

def load_inventory(filepath='inventory.yaml'):
    with open(filepath, 'r') as f:
        return yaml.safe_load(f)['devices']

def get_role_from_prompt(prompt):
    hostname = prompt.rstrip("#").strip()
    if hostname.startswith("R"):
        return "routeur", hostname
    elif "CORE" in hostname:
        return "core", hostname
    elif "ACC" in hostname:
        return "accs", hostname
    return "inconnu", hostname

def load_config_commands(hostname):
    path = f'configs/{hostname}.txt'
    if not os.path.exists(path):
        raise FileNotFoundError(f"Fichier de configuration introuvable : {path}")
    with open(path, "r", encoding="utf-8") as f:
        return [line.strip() for line in f if line.strip() and not line.strip().startswith("#")]

def deploy():
    os.makedirs("backups", exist_ok=True)
    os.makedirs("logs", exist_ok=True)
    devices = load_inventory()

    for dev in devices:
        ip = dev["host"]
        print(f"\n Connexion vers {ip}...")

Get Help Write Out Where Is Cut Text Justify Cur Pos Undo Mark Text To Bracket Previous Back Left Prev Word
Exit Read File Replace Paste Text To Spell Go To Line Redo Copy Text Where Was Next Forward Right Next Word
```

J'ai changé cette ligne 6 par la ligne suivante

```
GNU nano 4.8 src/main.py
import os
import yaml
import logging
from datetime import datetime
from netmiko import ConnectHandler, SSHDetect
from netmiko import NetmikoTimeoutException, NetmikoAuthenticationException

logging.basicConfig(
    filename='logs/deployment.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

def load_inventory(filepath='inventory.yaml'):
    with open(filepath, "r") as f:
        return yaml.safe_load(f)['devices']

def get_role_from_prompt(prompt):
    hostname = prompt.rstrip("#").strip()
    if hostname.startswith("R"):
        return "routeur", hostname
    elif "CORE" in hostname:
        return "core", hostname
    elif "ACC" in hostname:
        return "accs", hostname
    return "inconnu", hostname

def load_config_commands(hostname):
    path = f'configs/{hostname}.txt'
    if not os.path.exists(path):
        raise FileNotFoundError(f"Fichier de configuration introuvable : {path}")
    with open(path, "r", encoding="utf-8") as f:
        return [line.strip() for line in f if line.strip() and not line.strip().startswith("#")]

def deploy():
    os.makedirs("backups", exist_ok=True)
    os.makedirs("logs", exist_ok=True)
    devices = load_inventory()

    for dev in devices:
        ip = dev["host"]
        print(f"\n Connexion vers {ip}...")

Get Help Write Out Where Is Cut Text Justify Cur Pos Undo Mark Text To Bracket Previous Back Left Prev Word
Exit Read File Replace Paste Text To Spell Go To Line Redo Copy Text Where Was Next Forward Right Next Word
```

Je réexécute le script

```
root@NetworkAutomation-1:~/projet-devnet-campus# python3 src/main.py

Connexion vers 192.168.10.10...
Erreur sur 192.168.10.10 : 'device_type'

Connexion vers 192.168.10.20...
Erreur sur 192.168.10.20 : 'device_type'

Connexion vers 192.168.10.30...
Erreur sur 192.168.10.30 : 'device_type'

Connexion vers 192.168.10.40...
Erreur sur 192.168.10.40 : 'device_type'

Connexion vers 192.168.10.50...
Erreur sur 192.168.10.50 : 'device_type'

Connexion vers 192.168.10.60...
Erreur sur 192.168.10.60 : 'device_type'
root@NetworkAutomation-1:~/projet-devnet-campus# python3 src/main.py

Connexion vers 192.168.10.10...
Erreur sur 192.168.10.10 : 'device_type'

Connexion vers 192.168.10.20...
Erreur sur 192.168.10.20 : 'device_type'

Connexion vers 192.168.10.30...
Erreur sur 192.168.10.30 : 'device_type'

Connexion vers 192.168.10.40...
Erreur sur 192.168.10.40 : 'device_type'

Connexion vers 192.168.10.50...
Erreur sur 192.168.10.50 : 'device_type'

Connexion vers 192.168.10.60...
Erreur sur 192.168.10.60 : 'device_type'
```

Le problème de la ligne 6 est réglé mais cette fois-ci en réexécutant le script Il y a une autre erreur

L'erreur `KeyError: 'device_type'` indique que le dictionnaire d'équipement dans le script Python (ou dans votre fichier YAML/JSON de variables) manque de la clé `'device_type'`, dont Netmiko a obligatoirement besoin pour savoir quel pilote utiliser (`cisco_ios`, `cisco_xe`, etc.).

Mais moi je suis allé dans le fichier `inventory.yml` :

```
---
devices:
  - host: "192.168.10.10"
    username: "devnet"
    password: "cisco"
    secret: "cisco"
    device_type: "cisco_ios"
  - host: "192.168.10.20"
    username: "devnet"
    password: "cisco"
    secret: "cisco"
    device_type: "cisco_ios"
  - host: "192.168.10.30"
    username: "devnet"
    password: "cisco"
    secret: "cisco"
    device_type: "cisco_ios"
  - host: "192.168.10.40"
    username: "devnet"
    password: "cisco"
    secret: "cisco"
    device_type: "cisco_ios"
  - host: "192.168.10.50"
    username: "devnet"
    password: "cisco"
    secret: "cisco"
    device_type: "cisco_ios"
  - host: "192.168.10.60"
    username: "devnet"
    password: "cisco"
    secret: "cisco"
    device_type: "cisco_ios"
```

J'ai changé la devise_type en `cisco_ios` pour chaque équipement

```
root@NetworkAutomation-1:~/projet-devnet-campus# python3 src/main.py

Connexion vers 192.168.10.10...
Erreur sur 192.168.10.10 : The connection device_type must be 'autodetect'

Connexion vers 192.168.10.20...
Erreur sur 192.168.10.20 : The connection device_type must be 'autodetect'

Connexion vers 192.168.10.30...
Erreur sur 192.168.10.30 : The connection device_type must be 'autodetect'

Connexion vers 192.168.10.40...
Erreur sur 192.168.10.40 : The connection device_type must be 'autodetect'

Connexion vers 192.168.10.50...
Erreur sur 192.168.10.50 : The connection device_type must be 'autodetect'

Connexion vers 192.168.10.60...
Erreur sur 192.168.10.60 : The connection device_type must be 'autodetect'
```

J'ai réexécuté le script après quelques changements Mais je n'arrive toujours pas à attaquer les équipements et appliquer les instructions donner dans le script

L'erreur `The connection device_type must be 'autodetect'` indique que le code Python utilise le module `SSHDetect` de Netmiko, ce qui requiert obligatoirement que la valeur attribuée à `device_type` soit `'autodetect'` au lieu de `'cisco_ios'`.

```
GNU nano 4.8 inventory.yml
devices:
- host: "192.168.10.10"
  username: "devnet"
  password: "cisco"
  secret: "cisco"
  device_type: "autodetect"
- host: "192.168.10.20"
  username: "devnet"
  password: "cisco"
  secret: "cisco"
  device_type: "autodetect"
- host: "192.168.10.30"
  username: "devnet"
  password: "cisco"
  secret: "cisco"
  device_type: "autodetect"
- host: "192.168.10.40"
  username: "devnet"
  password: "cisco"
  secret: "cisco"
  device_type: "autodetect"
- host: "192.168.10.50"
  username: "devnet"
  password: "cisco"
  secret: "cisco"
  device_type: "autodetect"
- host: "192.168.10.60"
  username: "devnet"
  password: "cisco"
  secret: "cisco"
  device_type: "autodetect"

[Read 38 lines]
Get Help Write Out Where Is Cut Text Justify Cur Pos Undo Mark Text To Bracket Previous Back Left Prev Word
Exit Read File Replace Paste Text To Spell Go To Line Redo Copy Text Where Was Next Forward Right Next Word
```

Pour corriger ça je suis allé dans le fichier Inventory.yml puis J'ai remplacé « cisco_ios » par « autodetect »

```
root@NetworkAutomation-1:~/projet-devnet-campus# python3 src/main.py

Connexion vers 192.168.10.10...
Timeout / Équipement injoignable : 192.168.10.10

Connexion vers 192.168.10.20...
Timeout / Équipement injoignable : 192.168.10.20

Connexion vers 192.168.10.30...
Timeout / Équipement injoignable : 192.168.10.30

Connexion vers 192.168.10.40...
Timeout / Équipement injoignable : 192.168.10.40

Connexion vers 192.168.10.50...
Timeout / Équipement injoignable : 192.168.10.50

Connexion vers 192.168.10.60...
Timeout / Équipement injoignable : 192.168.10.60
```

Le message Timeout / Équipement injoignable signifie que Netmiko tente d'ouvrir une session **SSH (port 22)** sur les équipements, mais que le service SSH n'est soit pas encore activé sur les équipements, soit bloqué/fermé (par défaut sur Cisco, seul Telnet sur le port 23 ou la console est actif avant d'avoir généré les clés RSA).

```

R1-DKR#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R1-DKR(config)#ip domain name lab.local
R1-DKR(config)#crypto key generate rsa general-keys modulus 1024
The name for the keys will be: R1-DKR.lab.local

% The key modulus size is 1024 bits
% Generating 1024 bit RSA keys, keys will be non-exportable...
[OK] (elapsed time was 1 seconds)

R1-DKR(config)#line vty 0 4
R1-DKR(config-line)# transport input ssh telnet
R1-DKR(config-line)# login local
R1-DKR(config-line)#end
R1-DKR#write memory
*Sep  7 19:39:27.291: %SSH-5-ENABLED: SSH 1.99 has been enabled
*Sep  7 19:39:27.367: %SYS-5-CONFIG_I: Configured from console by console
R1-DKR#write memory
Building configuration...
[OK]
R1-DKR#

```

Sur chaque équipement j'ai exécuté ces commandes

5.9 Réexécution le script

```

root@NetworkAutomation-1:~/projet-devnet-campus# python3 src/main.py

Connexion vers 192.168.10.10...
OS détecté : cisco_ios
Équipement : R1-DKR | Rôle : routeur
Backup sauvegardé : backups/R1-DKR_20260907_194135.cfg
Configuration appliquée avec succès sur R1-DKR.

Connexion vers 192.168.10.20...
OS détecté : cisco_ios
Équipement : R2-ABJ | Rôle : routeur
Backup sauvegardé : backups/R2-ABJ_20260907_194146.cfg
Configuration appliquée avec succès sur R2-ABJ.

Connexion vers 192.168.10.30...
OS détecté : cisco_ios
Équipement : SW1-CORE | Rôle : core
Backup sauvegardé : backups/SW1-CORE_20260907_194157.cfg
Configuration appliquée avec succès sur SW1-CORE.

Connexion vers 192.168.10.40...
OS détecté : cisco_ios
Équipement : SW2-CORE | Rôle : core
Backup sauvegardé : backups/SW2-CORE_20260907_194208.cfg
Configuration appliquée avec succès sur SW2-CORE.

Connexion vers 192.168.10.50...
OS détecté : cisco_ios
Équipement : SW3-ACC | Rôle : acces
Backup sauvegardé : backups/SW3-ACC_20260907_194219.cfg
Configuration appliquée avec succès sur SW3-ACC.

Connexion vers 192.168.10.60...
OS détecté : cisco_ios
Équipement : SW4-ACC | Rôle : acces
Backup sauvegardé : backups/SW4-ACC_20260907_194230.cfg
Configuration appliquée avec succès sur SW4-ACC.
root@NetworkAutomation-1:~/projet-devnet-campus# nano configs/R1-DKR.txt

```

Et là le script est bien exécuté il se connecte sur chaque équipement et applique les configurations indiquées

6 Configurer PC1 (VLAN 10 - Dakar)

```

PC1> ip 10.10.10.10 255.255.255.0 10.10.10.1
Checking for duplicate address...
PC1 : 10.10.10.10 255.255.255.0 gateway 10.10.10.1

PC1> show ip

NAME       : PC1[1]
IP/MASK     : 10.10.10.10/24
GATEWAY     : 10.10.10.1
DNS         :
MAC         : 00:50:79:66:68:00
LPORT      : 10027
RHOST:PORT  : 127.0.0.1:10028
MTU         : 1500

```

J'ai appliqué ces 2 commandes sur la console du PC1, la première commande c'est l'attribution d'une adresse IP au PC1 le 10.10.10.10 et son gateway qui est le 10.10.10.1 L'adresse du routeur R1-DKR

Et la 2^e commande (show ip) c'est une vérification si l'adresse a été bien attribuée à l'équipement

6.1 Tester la connectivité locale (Passerelles)

```

PC1> ping 10.10.10.1
host (10.10.10.1) not reachable

```

Depuis le PC1 Je vérifie l'accès à la passerelle sur R1-DKR mais le ping ne passe pas

```

R1-DKR#show ip interface brief
Interface      IP-Address      OK? Method Status      Protocol
FastEthernet0/0 192.168.20.1   YES manual up          up
GigabitEthernet1/0 192.168.10.10 YES manual up          up
Serial2/0        unassigned     YES unset  administratively down down
Serial2/1        unassigned     YES unset  administratively down down
Serial2/2        unassigned     YES unset  administratively down down
Serial2/3        unassigned     YES unset  administratively down down
Serial2/4        unassigned     YES unset  administratively down down
Serial2/5        unassigned     YES unset  administratively down down
Serial2/6        unassigned     YES unset  administratively down down
Serial2/7        unassigned     YES unset  administratively down down
Serial3/0        unassigned     YES unset  administratively down down
Serial3/1        unassigned     YES unset  administratively down down
Serial3/2        unassigned     YES unset  administratively down down
Serial3/3        unassigned     YES unset  administratively down down
Ethernet4/0      unassigned     YES unset  administratively down down
Ethernet4/1      unassigned     YES unset  administratively down down
Ethernet4/2      unassigned     YES unset  administratively down down
Ethernet4/3      unassigned     YES unset  administratively down down
FastEthernet6/0  unassigned     YES unset  administratively down down
R1-DKR#

```

Je suis allé vérifier les interfaces du routeur s'ils ont été bien attribués et qu'ils sont en up

L'interface parent **Ethernet4/0** (ainsi que la sous-interface `Ethernet4/0.10`) est en état **administratively down**, et les commandes envoyées par le script n'ont pas activé le port parent.

Mais je me suis rendu compte que les scripts a passé mais les instructions ne sont pas appliquées à cause de J'ai pas ajouté la ligne pour qu'il rentre d'abord dans les configurations globales pour appliquer les commandes alors que Netmiko est connecté en mode privilège donc l'équipement rejette les commandes comme erreurs de syntaxe.

```
GNU nano 4.8                                configs/R1-DKR.txt
configure terminal
hostname R1-DKR
!
interface Ethernet4/0
description Link to SM1-CORE
no shutdown
!
interface Ethernet4/0.10
description Gateway LAN DKR
encapsulation dot1Q 10
ip address 10.10.10.1 255.255.255.0
no shutdown
!
interface FastEthernet0/0
description WAN Link to R2-ABJ
ip address 192.168.20.1 255.255.255.252
no shutdown
!
router eigrp 100
network 10.10.10.0 0.0.0.255
network 192.168.20.0 0.0.0.3
no auto-summary
end

GNU nano 4.8                                configs/R2-ABJ.txt
configure terminal
hostname R2-ABJ
!
interface Ethernet4/0
description Link to SM2-CORE
no shutdown
!
interface Ethernet4/0.20
description Gateway LAN ABJ
encapsulation dot1Q 20
ip address 10.20.10.1 255.255.255.0
no shutdown
!
interface FastEthernet0/0
description WAN Link to R1-DKR
ip address 192.168.20.2 255.255.255.252
no shutdown
!
router eigrp 100
network 10.20.10.0 0.0.0.255
network 192.168.20.0 0.0.0.3
no auto-summary
end
```

J'ai ajouté cette ligne « configure terminal » dans les fichiers txt de R1-DKR, R2-ABJ et sur tous les autres fichiers

```
root@NetworkAutomation-1:~/projet-devnet-campus# python3 src/main.py
```

```
Connexion vers 192.168.10.10...  
OS détecté : cisco_ios  
Équipement : R1-DKR | Rôle : routeur  
Backup sauvegardé : backups/R1-DKR_20260907_195911.cfg  
Configuration appliquée avec succès sur R1-DKR.
```

```
Connexion vers 192.168.10.20...  
OS détecté : cisco_ios  
Équipement : R2-ABJ | Rôle : routeur  
Backup sauvegardé : backups/R2-ABJ_20260907_195923.cfg  
Configuration appliquée avec succès sur R2-ABJ.
```

```
Connexion vers 192.168.10.30...  
OS détecté : cisco_ios  
Équipement : SW1-CORE | Rôle : core  
Backup sauvegardé : backups/SW1-CORE_20260907_195934.cfg  
Configuration appliquée avec succès sur SW1-CORE.
```

```
Connexion vers 192.168.10.40...  
OS détecté : cisco_ios  
Équipement : SW2-CORE | Rôle : core  
Backup sauvegardé : backups/SW2-CORE_20260907_195945.cfg  
Configuration appliquée avec succès sur SW2-CORE.
```

```
Connexion vers 192.168.10.50...  
OS détecté : cisco_ios  
Équipement : SW3-ACC | Rôle : acces  
Backup sauvegardé : backups/SW3-ACC_20260907_195956.cfg  
Configuration appliquée avec succès sur SW3-ACC.
```

```
Connexion vers 192.168.10.60...  
OS détecté : cisco_ios  
Équipement : SW4-ACC | Rôle : acces  
Backup sauvegardé : backups/SW4-ACC_20260907_200007.cfg  
Configuration appliquée avec succès sur SW4-ACC.
```

J'ai relancé le script Ensuite je suis retourné faire le Ping de PC envers la passerelle du R1-DKR

```
PC1> ping 10.10.10.1  
84 bytes from 10.10.10.1 icmp_seq=1 ttl=255 time=15.576 ms  
84 bytes from 10.10.10.1 icmp_seq=2 ttl=255 time=16.437 ms  
84 bytes from 10.10.10.1 icmp_seq=3 ttl=255 time=15.804 ms  
84 bytes from 10.10.10.1 icmp_seq=4 ttl=255 time=16.319 ms  
84 bytes from 10.10.10.1 icmp_seq=5 ttl=255 time=16.360 ms
```

Le Ping passe et les 2 équipements arrivent à se rejoindre

6.2 Tester de la connectivité globale

Depuis le PC1 j'ai fait un Ping vers le PC2 mais les deux équipements n'arrivent pas à se rejoindre pour le moment

```
PC1> ping 10.20.10.10
*10.10.10.1 icmp_seq=1 ttl=255 time=16.581 ms (ICMP type:3, code:1, Destination host unreachable)
*10.10.10.1 icmp_seq=2 ttl=255 time=16.130 ms (ICMP type:3, code:1, Destination host unreachable)
*10.10.10.1 icmp_seq=3 ttl=255 time=16.618 ms (ICMP type:3, code:1, Destination host unreachable)
*10.10.10.1 icmp_seq=4 ttl=255 time=16.281 ms (ICMP type:3, code:1, Destination host unreachable)
*10.10.10.1 icmp_seq=5 ttl=255 time=16.339 ms (ICMP type:3, code:1, Destination host unreachable)
```

Le message `Destination host unreachable` indique que **R1-DKR ne connaît pas le réseau destination 10.20.10.0/24** dans sa table de routage, ou que la sous-interface vers R2-ABJ / le voisinage EIGRP est hors service.

J'ai exécuté la commande suivante sur **R1-DKR** pour vérifier si la route vers Abidjan (10.20.10.0/24) est présente via EIGRP :

```
R1-DKR#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       + - replicated route, % - next hop override

Gateway of last resort is not set

    10.0.0.0/8 is variably subnetted, 3 subnets, 3 masks
D    10.0.0.0/8 is a summary, 00:15:00, Null0
C    10.10.10.0/24 is directly connected, Ethernet4/0.10
L    10.10.10.1/32 is directly connected, Ethernet4/0.10
    192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C    192.168.10.0/24 is directly connected, GigabitEthernet1/0
L    192.168.10.10/32 is directly connected, GigabitEthernet1/0
    192.168.20.0/24 is variably subnetted, 3 subnets, 3 masks
D    192.168.20.0/24 is a summary, 00:15:17, Null0
C    192.168.20.0/30 is directly connected, FastEthernet0/0
L    192.168.20.1/32 is directly connected, FastEthernet0/0
```

```
R2-ABJ#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       + - replicated route, % - next hop override

Gateway of last resort is not set

    10.0.0.0/8 is variably subnetted, 3 subnets, 3 masks
D    10.0.0.0/8 is a summary, 00:16:26, Null0
C    10.20.10.0/24 is directly connected, Ethernet4/0.20
L    10.20.10.1/32 is directly connected, Ethernet4/0.20
    192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C    192.168.10.0/24 is directly connected, GigabitEthernet1/0
L    192.168.10.20/32 is directly connected, GigabitEthernet1/0
    192.168.20.0/24 is variably subnetted, 3 subnets, 3 masks
D    192.168.20.0/24 is a summary, 00:16:29, Null0
C    192.168.20.0/30 is directly connected, FastEthernet0/0
L    192.168.20.2/32 is directly connected, FastEthernet0/0
```

On voit les 2 réseaux qu'on avait indiqués dans les fichiers de txt R1-DKR et R2-ABJ qui utilisent le routage dynamique EIGRP

Mais problème apparaît très clairement dans les tables de routage : **l'auto-sommarisation d'EIGRP (auto-summary) est activée.**

Sur les deux routeurs, EIGRP résume le réseau 10.0.0.0/8 vers Null0 (D 10.0.0.0/8 is a summary, Null0). Comme les deux sites utilisent des sous-réseaux appartenant au même réseau classe A (10.10.10.0/24 à Dakar et 10.20.10.0/24 à Abidjan), la sommarisation automatique empêche l'échange des sous-réseaux spécifiques /24 à travers le lien WAN (192.168.20.0/30).

Mise à jour des fichiers de configuration .txt

J'ai remplacé `auto-summary` par `no auto-summary` dans les fichiers `configs/R1-DKR.txt` et `configs/R2-ABJ.txt` pour éviter que le problème ne se reproduise au prochain lancement du script.

Sur R1-DKR

```
GNU nano 4.8                                     configs/R1-DKR.txt
configure terminal
hostname R1-DKR
!
interface Ethernet4/0
description Link to SW1-CORE
no shutdown
!
interface Ethernet4/0.10
description Gateway LAN DKR
encapsulation dot1Q 10
ip address 10.10.10.1 255.255.255.0
no shutdown
!
interface FastEthernet0/0
description WAN Link to R2-ABJ
ip address 192.168.20.1 255.255.255.252
no shutdown
!
router eigrp 100
network 10.10.10.0 0.0.0.255
network 192.168.20.0 0.0.0.3
no auto-summary
end
```

Sur R2-ABJ

```
GNU nano 4.8                                     configs/R2-ABJ.txt
configure terminal
hostname R2-ABJ
!
interface Ethernet4/0
description Link to SW2-CORE
no shutdown
!
interface Ethernet4/0.20
description Gateway LAN ABJ
encapsulation dot1Q 20
ip address 10.20.10.1 255.255.255.0
no shutdown
!
interface FastEthernet0/0
description WAN Link to R1-DKR
ip address 192.168.20.2 255.255.255.252
no shutdown
!
router eigrp 100
network 10.20.10.0 0.0.0.255
network 192.168.20.0 0.0.0.3
no auto-summary
end
```

Puis J'ai réexécuté le script

```
root@NetworkAutomation-1:~/projet-devnet-campus# python3 src/main.py
```

```
Connexion vers 192.168.10.10...
```

```
OS détecté : cisco_ios
```

```
Équipement : R1-DKR | Rôle : routeur
```

```
Backup sauvegardé : backups/R1-DKR_20260907_204035.cfg
```

```
Configuration appliquée avec succès sur R1-DKR.
```

```
Connexion vers 192.168.10.20...
```

```
OS détecté : cisco_ios
```

```
Équipement : R2-ABJ | Rôle : routeur
```

```
Backup sauvegardé : backups/R2-ABJ_20260907_204047.cfg
```

```
Configuration appliquée avec succès sur R2-ABJ.
```

```
Connexion vers 192.168.10.30...
```

```
OS détecté : cisco_ios
```

```
Équipement : SW1-CORE | Rôle : core
```

```
Backup sauvegardé : backups/SW1-CORE_20260907_204059.cfg
```

```
Configuration appliquée avec succès sur SW1-CORE.
```

```
Connexion vers 192.168.10.40...
```

```
OS détecté : cisco_ios
```

```
Équipement : SW2-CORE | Rôle : core
```

```
Backup sauvegardé : backups/SW2-CORE_20260907_204110.cfg
```

```
Configuration appliquée avec succès sur SW2-CORE.
```

```
Connexion vers 192.168.10.50...
```

```
OS détecté : cisco_ios
```

```
Équipement : SW3-ACC | Rôle : acces
```

```
Backup sauvegardé : backups/SW3-ACC_20260907_204121.cfg
```

```
Configuration appliquée avec succès sur SW3-ACC.
```

```
Connexion vers 192.168.10.60...
```

```
OS détecté : cisco_ios
```

```
Équipement : SW4-ACC | Rôle : acces
```

```
Backup sauvegardé : backups/SW4-ACC_20260907_204132.cfg
```

```
Configuration appliquée avec succès sur SW4-ACC.
```

6.3 Vérification de ping

```
PC1> ping 10.20.10.10
```

```
84 bytes from 10.20.10.10 icmp_seq=1 ttl=62 time=93.163 ms
```

```
84 bytes from 10.20.10.10 icmp_seq=2 ttl=62 time=62.168 ms
```

```
84 bytes from 10.20.10.10 icmp_seq=3 ttl=62 time=62.540 ms
```

```
84 bytes from 10.20.10.10 icmp_seq=4 ttl=62 time=61.913 ms
```

```
84 bytes from 10.20.10.10 icmp_seq=5 ttl=62 time=62.774 ms
```

```
PC1> █
```

```
PC2> ping 10.10.10.10
84 bytes from 10.10.10.10 icmp_seq=1 ttl=62 time=62.266 ms
84 bytes from 10.10.10.10 icmp_seq=2 ttl=62 time=62.179 ms
84 bytes from 10.10.10.10 icmp_seq=3 ttl=62 time=62.702 ms
84 bytes from 10.10.10.10 icmp_seq=4 ttl=62 time=63.094 ms
84 bytes from 10.10.10.10 icmp_seq=5 ttl=62 time=62.475 ms

PC2> █
```

Après l'application des correctifs sur les équipements (activation de l'encapsulation Dot1Q sur les trunks et désactivation du `auto-summary` dans le processus EIGRP 100), la connectivité réseau a été vérifiée à l'aide de l'outil ICMP.

1. **Validation du flux Dakar -> Abidjan :** La commande `ping 10.20.10.10` exécutée depuis le poste **PC1** confirme que la passerelle **R1-DKR** relaye correctement les paquets à travers le lien WAN via la route EIGRP apprise dynamique (10.20.10.0/24). Les 5 paquets ICMP ont été transmis avec un temps de réponse moyen de 62 ms et une valeur TTL de 62 (reflétant le franchissement des 2 routeurs intermédiaires).
2. **Validation du flux retour Abidjan -> Dakar :** Le test inverse depuis le poste **PC2** vers l'adresse **10.10.10.10** s'est achevé avec le même succès (0% de perte de paquets).

Conclusion

La mise en place de ce processus d'automatisation a permis de valider avec succès le déploiement de l'infrastructure réseau. L'utilisation du script Python (`main.py`) a offert un gain de temps significatif tout en éliminant les risques d'erreurs de saisie manuelle. Chaque équipement a pu être identifié, sauvegardé et configuré de manière totalement homogène et sécurisée.

Les tests de recette effectués par requêtes ICMP (`ping`) entre les hôtes distants (PC1 à Dakar et PC2 à Abidjan) confirment la parfaite opérationnalité du réseau : le routage inter-VLAN, l'encapsulation Trunk 802.1Q et l'interconnexion WAN EIGRP acheminent correctement le trafic bidirectionnel. Ce projet démontre l'efficacité de l'approche *Infrastructure as Code* (IaC) appliquée aux réseaux, offrant une base solide et évolutive pour la gestion et la maintenance future du parc informatique.